
전자항법과 GPS

11주차

공간정보 프로그래밍 실습

임 호 용

imyong7@hanmail.net



전자항법과 GPS

1. 공간정보 프로그래밍

1-1. 파이썬 in QGIS

1-2. 파이썬으로 QGIS 핸들링하기

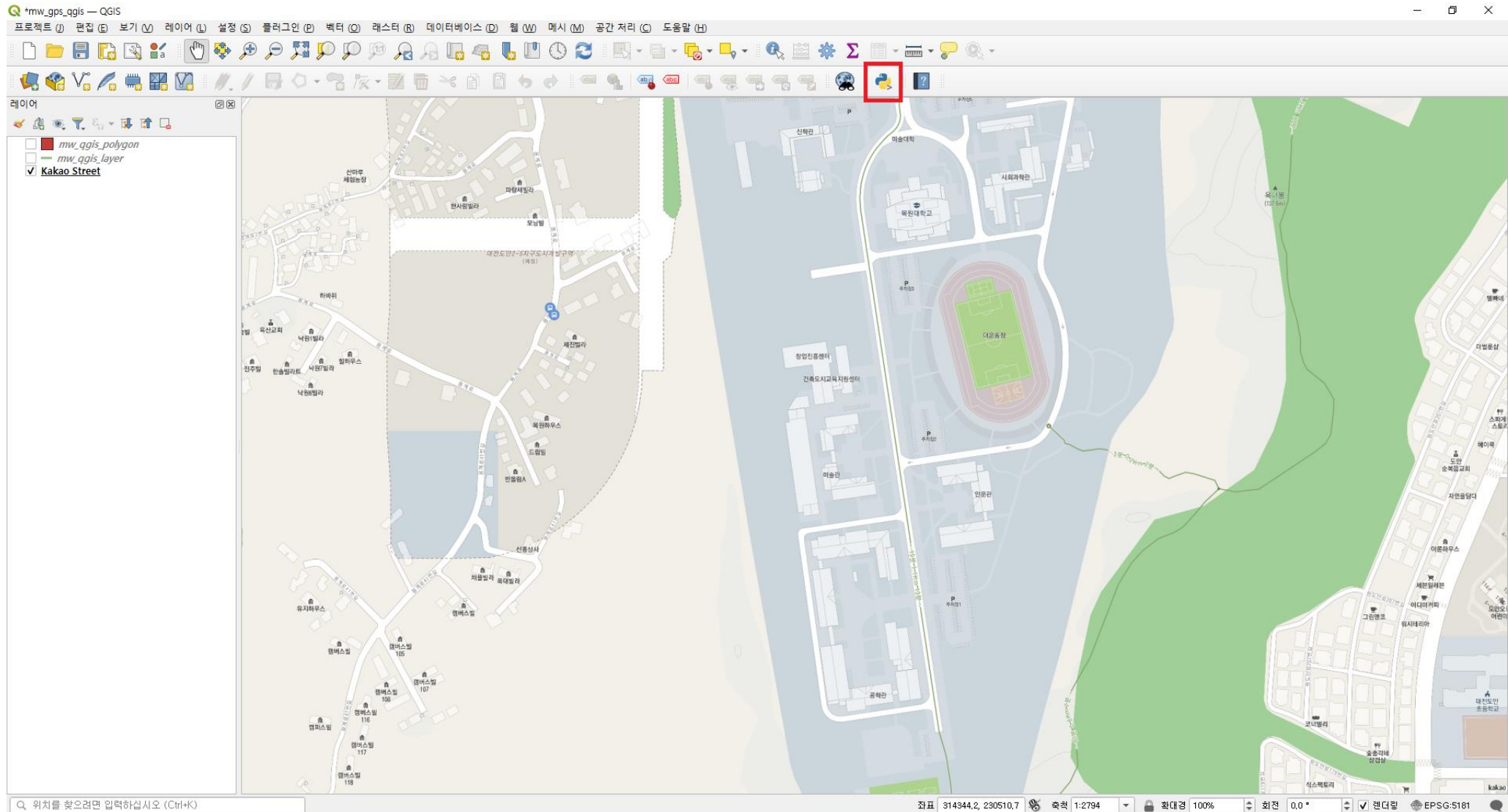
전자항법과 GPS

1. 공간정보 프로그래밍

1-1. 파이썬 in QGIS

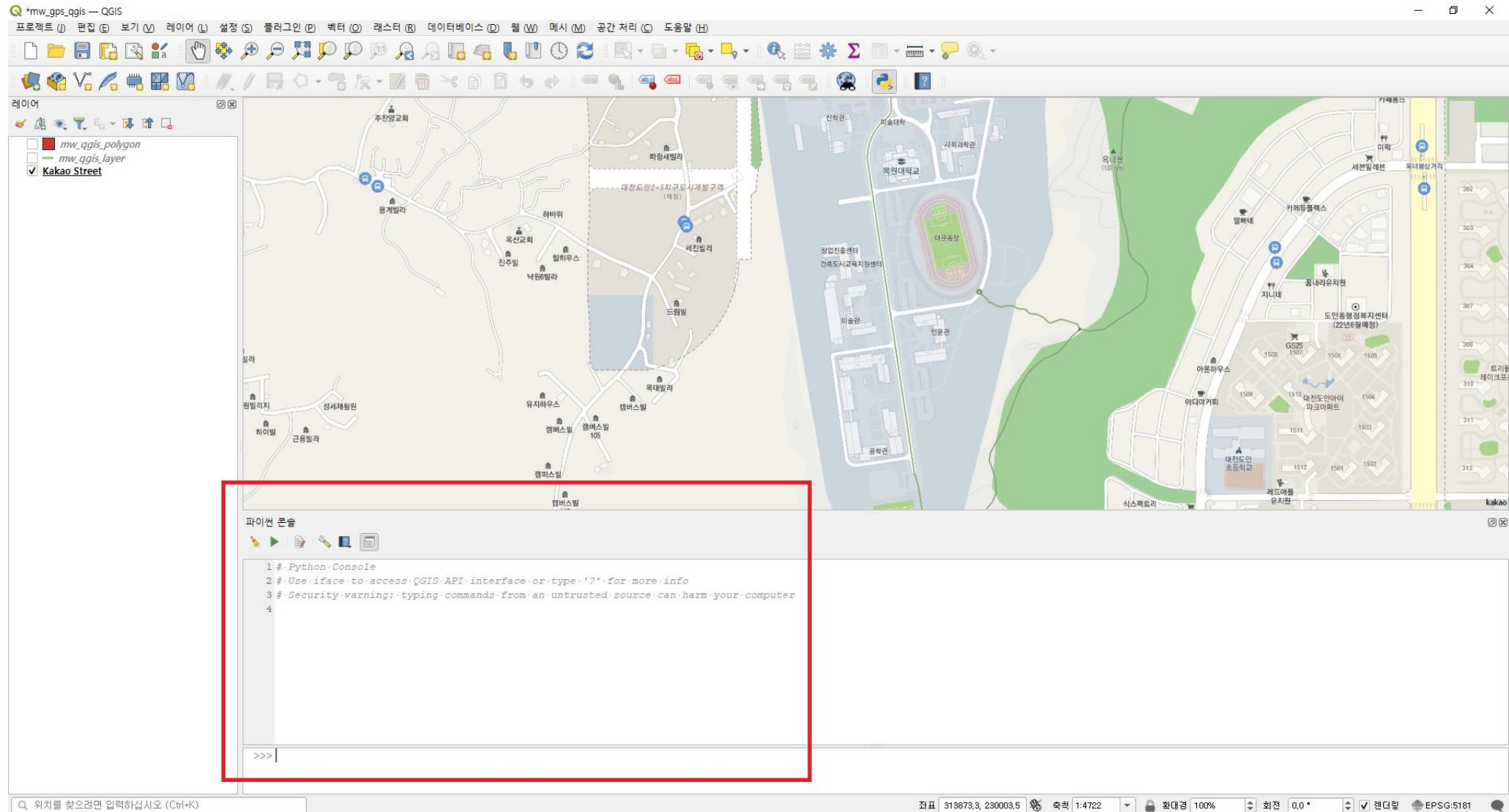
1. 공간정보 프로그래밍

파이썬 in QGIS



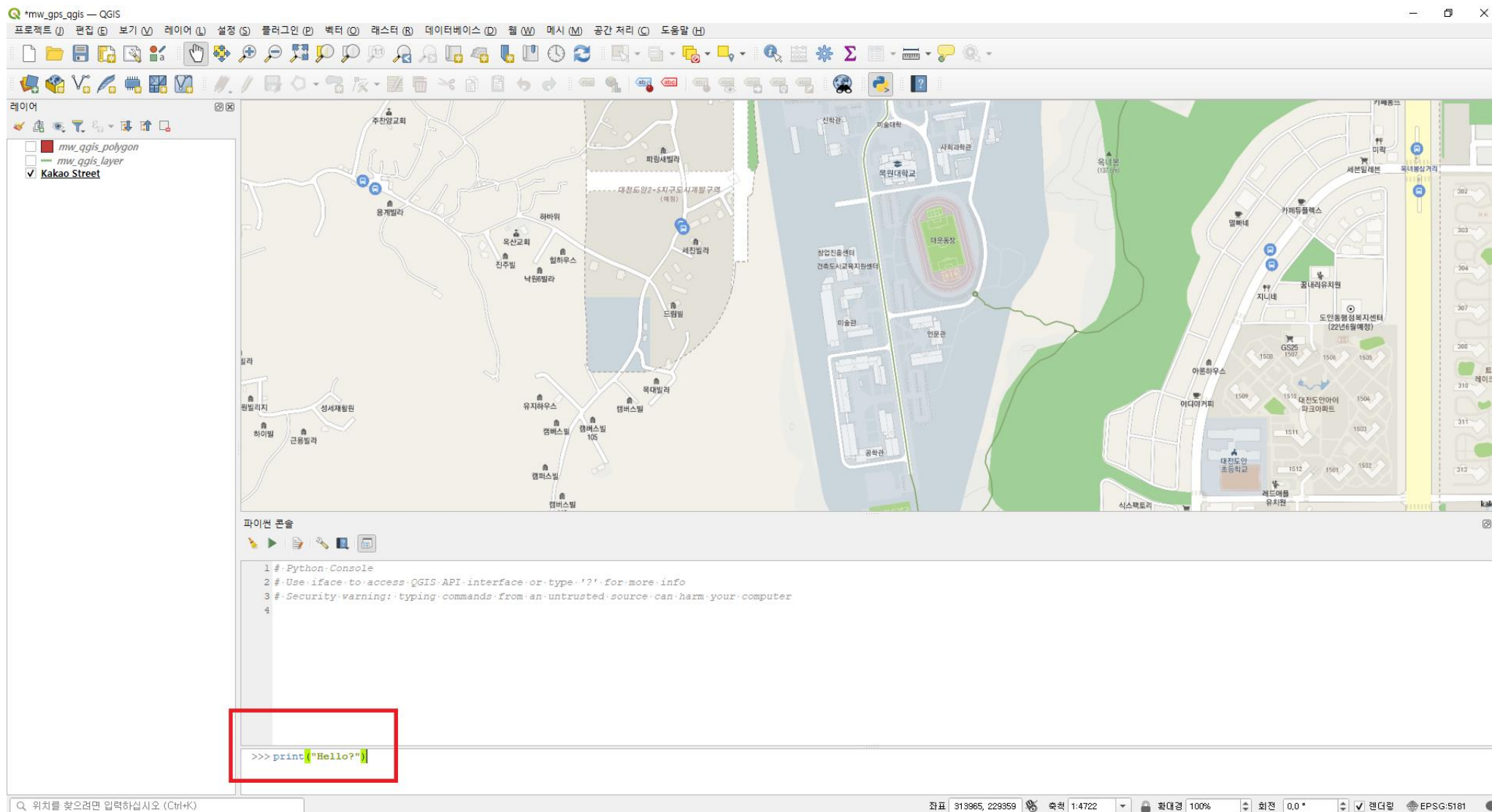
1. 공간정보 프로그래밍

파이썬 in QGIS



1. 공간정보 프로그래밍

파이썬 in QGIS



1. 공간정보 프로그래밍

파이썬 in QGIS

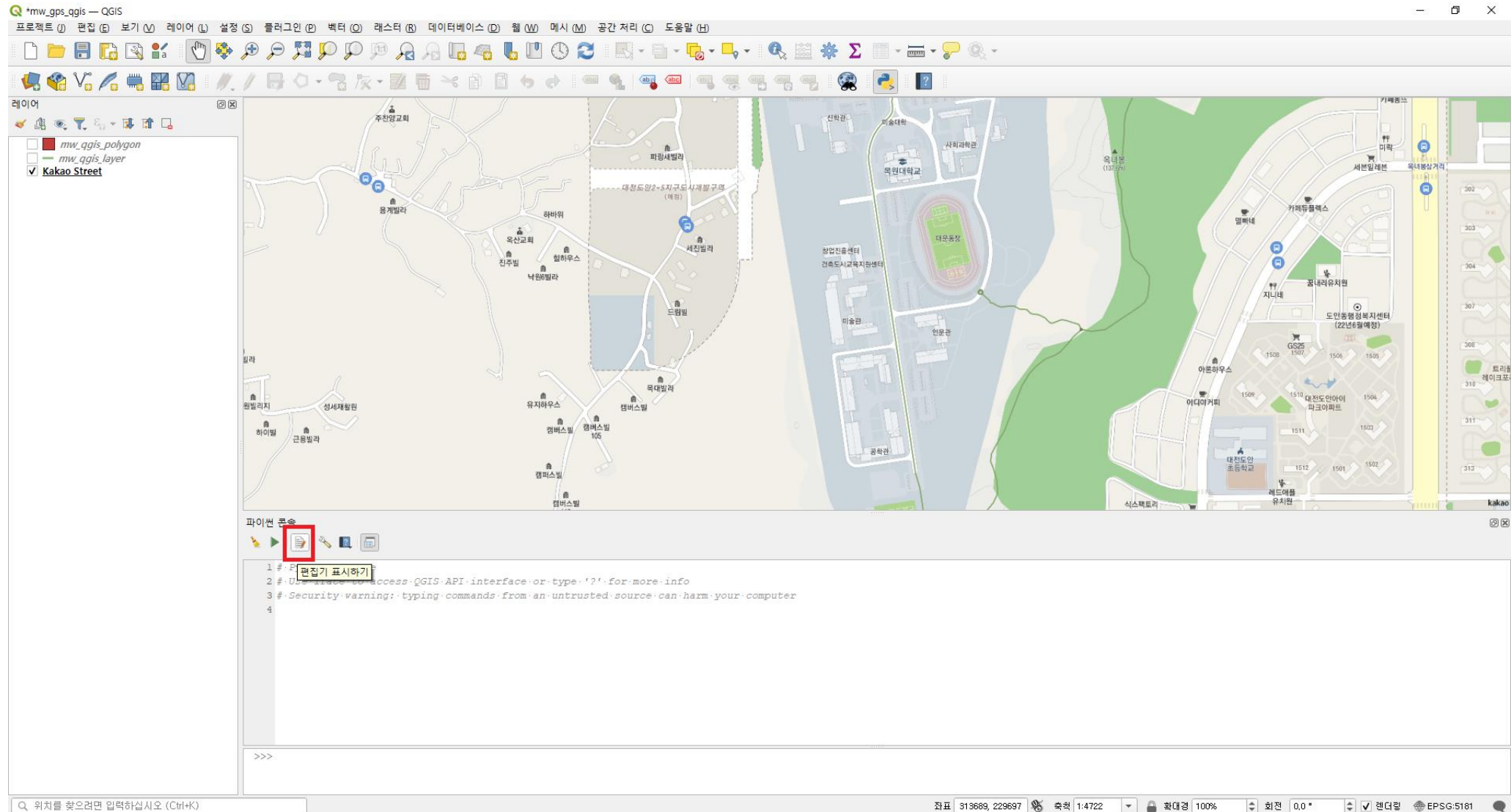
The screenshot displays the QGIS desktop application. The main map area shows a satellite view of Mokwon University and the surrounding urban area. The left sidebar contains a layer list with three items: 'mw_qgis_polygon' (red square), 'mw_qgis_layer' (green line), and 'Kakao Street' (checked). The bottom panel is the Python Console, which contains the following code:

```
1 # Python Console
2 # Use this to access QGIS API interface or type '?' for more info
3 # Security warning: typing commands from an untrusted source can harm your computer
4 >>> print("Hello?")
5 Hello?
6
```

The console output shows 'Hello?' on the line following the print command. The status bar at the bottom indicates the current coordinates (313689, 229644), scale (1:4722), and other settings.

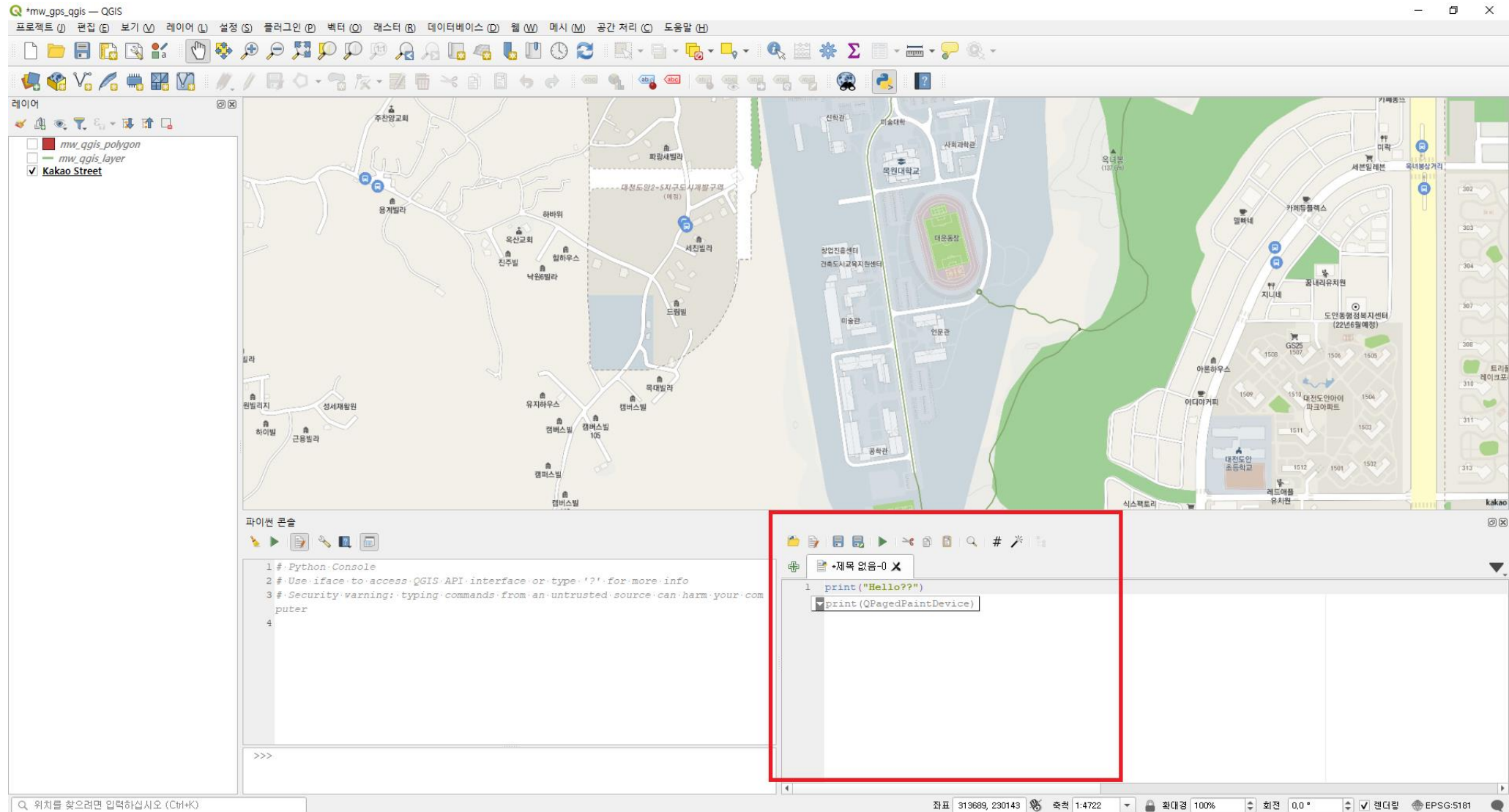
1. 공간정보 프로그래밍

파이썬 in QGIS



1. 공간정보 프로그래밍

파이썬 in QGIS



1. 공간정보 프로그래밍

파이썬 in QGIS

The screenshot displays the QGIS desktop application. The main map area shows a satellite view of Mokwon University and the surrounding area, with various landmarks and roads visible. The layer panel on the left lists three layers: 'mw_qgis_polygon' (red square), 'mw_qgis_layer' (green line), and 'Kakao Street' (checked). The Python console at the bottom is open, showing a list of instructions for using the console. A red box highlights the 'Run' button (a green play icon) in the console toolbar. The console contains the following code:

```
1 # Python Console
2 # Use iface to access QGIS API interface or type '?' for more info
3 # Security warning: typing commands from an untrusted source can harm your computer
4
5 print("Hello??")
```

The status bar at the bottom indicates the current coordinates (319830, 229359), scale (1:4722), and projection (EPSG:5181).

1. 공간정보 프로그래밍

파이썬 in QGIS

The screenshot displays the QGIS desktop environment. The top menu bar includes options like '프로젝트 (P)', '편집 (E)', '보기 (V)', '레이어 (L)', '설정 (S)', '플러그인 (P)', '벡터 (Q)', '래스터 (R)', '데이터베이스 (D)', '웹 (W)', '메시 (M)', '공간 처리 (C)', and '도움말 (H)'. The left sidebar shows the '레이어' (Layers) panel with three items: 'mw_qgis_polygon', 'mw_qgis_layer', and 'Kakao Street'. The main map area shows a street map of Mokwon University and its surroundings, with labels in Korean. The bottom panel is divided into two sections. The left section is the '파이썬 콘솔' (Python Console), which contains a script that reads a file and prints 'Hello??'. The right section is a file explorer showing a file named '재목 없음-0'. The status bar at the bottom indicates the current coordinates (319830, 229359), scale (1:4722), and projection (EPSG:5181).

```
1 # Python Console
2 # Use iface to access QGIS API interface on the fly. (2) for more info
3 # Security warning: typing commands from an untrusted source can harm your computer
4 >>> .exec(compile(Path('C:/Users/HOD/AppData/Local/Temp/tmp7it9ynkf').read_text(), '재목 없음-0', 'exec'))
5 Hello??
6
```

1. 공간정보 프로그래밍

파이썬 in QGIS

The screenshot displays the QGIS 3.16.16 interface. The main map area shows a street map of Mokwon University and its surroundings, including a large green field and several buildings. The left sidebar shows the 'Layers' panel with three layers: 'mw_qgis_polygon' (red square), 'mw_qgis_layer' (green line), and 'Kakao Street' (checked). The bottom panel is divided into two sections: the 'Python Console' on the left and the 'Command Line' on the right. The Python Console shows a red box around the prompt '>>>'. The Command Line shows the command 'print('Hello??')'.

Python Console:

```
1 # Python Console
2 # Use iface to access QGIS API interface or type '?' for more info
3 # Security warning: typing commands from an untrusted source can harm your computer
4
5 >>>
```

Command Line:

```
1 print('Hello??')
```


1. 공간정보 프로그래밍

파이썬 in QGIS

The screenshot displays the QGIS desktop application. The main map area shows a satellite view of Mokwon University and the surrounding area, with various landmarks and roads visible. The left sidebar contains a layer list with three items: 'mw_qgis_polygon' (red square), 'mw_qgis_layer' (green line), and 'Kakao Street' (checked, blue line). The bottom-left console window is open, showing the following text:

```
4 >>> .?
5 QGIS 파이썬 콘솔
6 -----
7
8 이 콘솔은 사용자가 파이썬 명령어를 실행할 수 있는 파이썬 인터프리터입니다.
9 QGIS (분석, 핵심, GUI, 3D) 와 Qt (QtCore, QtGui, QtNetwork, QtWidgets, QtXml)
10 모듈은 물론 파이썬의 math, os, re 및 sys 모듈까지 이미 임포트되어 있어서
11 바로 사용할 수 있습니다.
12
13 유용한 변수:
14
15 >>>
```

The bottom-right script editor window is also open, showing a Python script:

```
1 print("Hello??")
```

The status bar at the bottom indicates the current map coordinates (313725, 229790), scale (1:4196), and projection (EPSG:5181).

1. 공간정보 프로그래밍

파이썬 in QGIS

The screenshot displays the QGIS desktop application. The main map window shows a street map of Mokwon University and the surrounding area, with labels in Korean. The left sidebar contains a layer list with three items: 'mw_qgis_polygon' (red square), 'mw_qgis_layer' (green line), and 'Kakao Street' (checked). The bottom panel is divided into two sections. The left section is the 'Python Console', which contains a list of commands. The first command, `exec(compile(Path('C:/Users/HOD/AppData/Local/Temp/tmp5sh2q9as').read_text(), 'C:\Program Files\Python\Python313\python.exe', 'exec'))`, is highlighted with a red box. The right section is a command prompt window, which shows the output of the first command: `print(math.fabs(-1))`, also highlighted with a red box. The status bar at the bottom indicates the current map coordinates (313763, 229473), scale (1:4196), and projection (EPSG:5181).

전자항법과 GPS

1. 공간정보 프로그래밍

1-2. 파이썬으로 QGIS 핸들링하기

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 리버스 지오코딩

The screenshot displays the QGIS desktop application. The main map window shows a street map of Mokwon University with a green line indicating a path. The left sidebar contains a layer list with 'mw_qgis_polygon', 'mw_qgis_layer', and 'Kakao Street' (checked). Below the map is the 'Python Console' tab, which contains the following code:

```
1 # Python Console
2 # Use iface to access QGIS API interface or type '?' for more info
3 # Security warning: typing commands from an untrusted source can harm your computer
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
```

The code defines a function to reverse geocode coordinates using the Kakao API. The API key is stored in a variable, and the function returns the address or None if the key is not set.

```
from qgis.PyQt.QtCore import Qt
from qgis.PyQt.QtGui import QCursor
from qgis.gui import QgsMapToolEmitPoint
from qgis.core import QgsProject, QgsCoordinateReferenceSystem, QgsCoordinateTransform
import urllib.request, json

KAKAO_REST_API_KEY = "KakaoAK....."

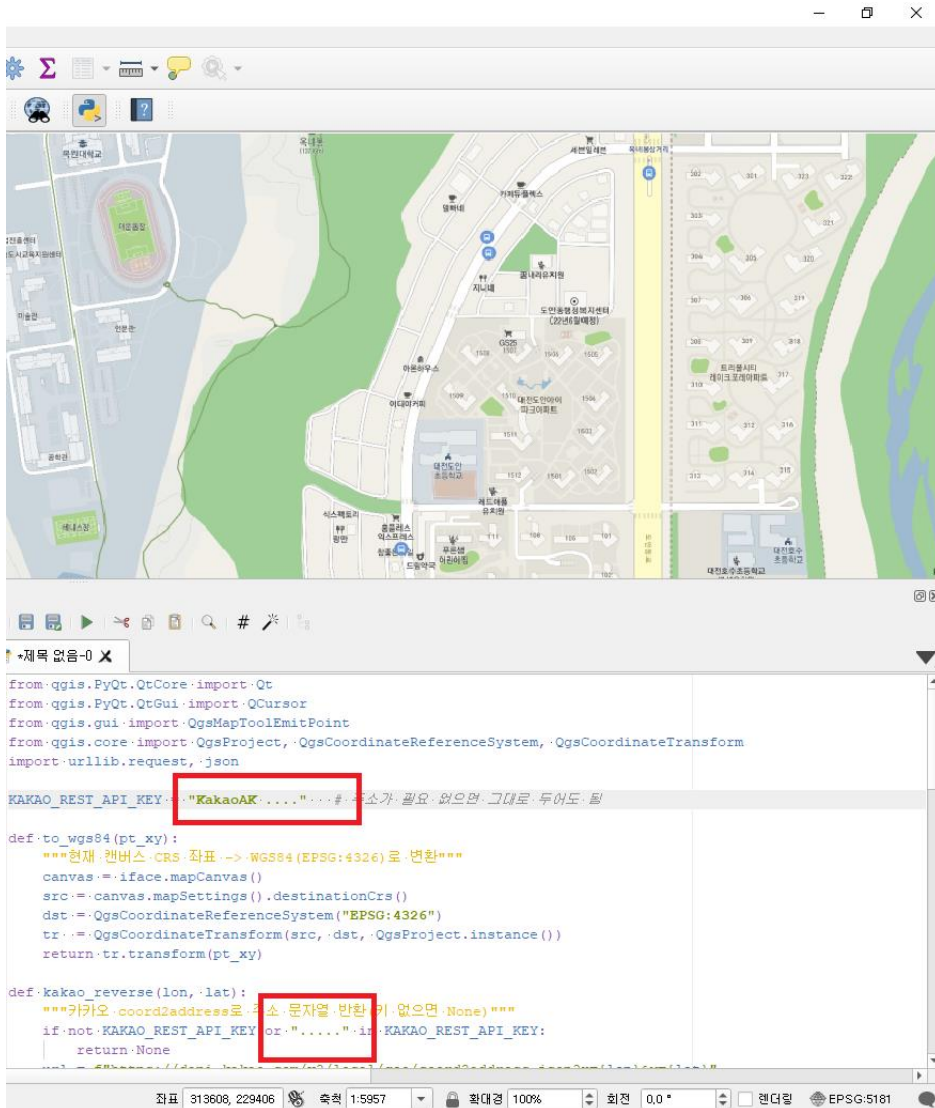
def to_wgs84(pt_xy):
    """현재 캔버스 CRS 좌표 -> WGS84 (EPSG:4326) 로 변환"""
    canvas = iface.mapCanvas()
    src = canvas.mapSettings().destinationCrs()
    dst = QgsCoordinateReferenceSystem("EPSG:4326")
    tr = QgsCoordinateTransform(src, dst, QgsProject.instance())
    return tr.transform(pt_xy)

def kakao_reverse(lon, lat):
    """카카오 coord2address로 주소 문자열 반환 (이 없으면 None)"""
    if not KAKAO_REST_API_KEY or "....." in KAKAO_REST_API_KEY:
        return None
    url = f"https://dapi.kakao.com/v3/local/address/geo2address?x={lon}&y={lat}&key={KAKAO_REST_API_KEY}"
```

The status bar at the bottom shows the coordinates 313608, 229406, a scale of 1:5957, a zoom level of 100%, and the coordinate system EPSG:5181.

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 리버스 지오코딩



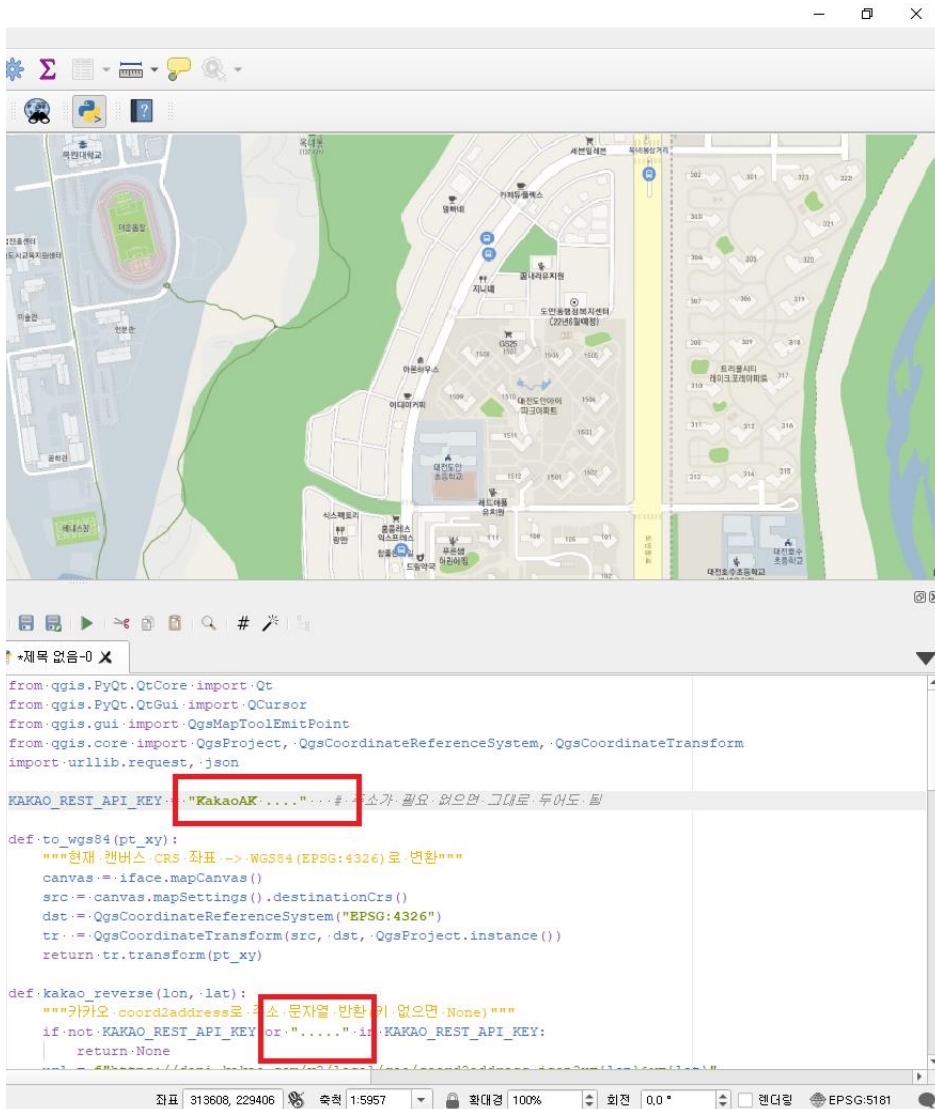
```

from qgis.PyQt.QtCore import Qt
from qgis.PyQt.QtGui import QCursor
from qgis.gui import QgsMapToolEmitPoint
from qgis.core import QgsProject,
QgsCoordinateReferenceSystem, QgsCoordinateTransform
import urllib.request, json
  
```

```
KAKAO_REST_API_KEY = "KakaoAK ..."
```

```

def to_wgs84(pt_xy):
    """현재 캔버스 CRS 좌표 -> WGS84(EPSSG:4326)로 변환"""
    canvas = iface.mapCanvas()
    src = canvas.mapSettings().destinationCrs()
    dst = QgsCoordinateReferenceSystem("EPSG:4326")
    tr = QgsCoordinateTransform(src, dst, QgsProject.instance())
    return tr.transform(pt_xy)
  
```



```
def kakao_reverse(lon, lat):
```

```
    """카카오 coord2address로 주소 문자열 반환(키 없으면 None)"""
```

```
    if not KAKAO_REST_API_KEY or "..." in KAKAO_REST_API_KEY:
        return None
```

```
    url = f"https://dapi.kakao.com/v2/local/geo/coord2addresses.json?x={lon}&y={lat}"
```

```
    req = urllib.request.Request(url, headers={"Authorization": KAKAO_REST_API_KEY})
```

```
    data = json.loads(urllib.request.urlopen(req, timeout=5).read().decode("utf-8"))
```

```
    docs = data.get("documents", [])
```

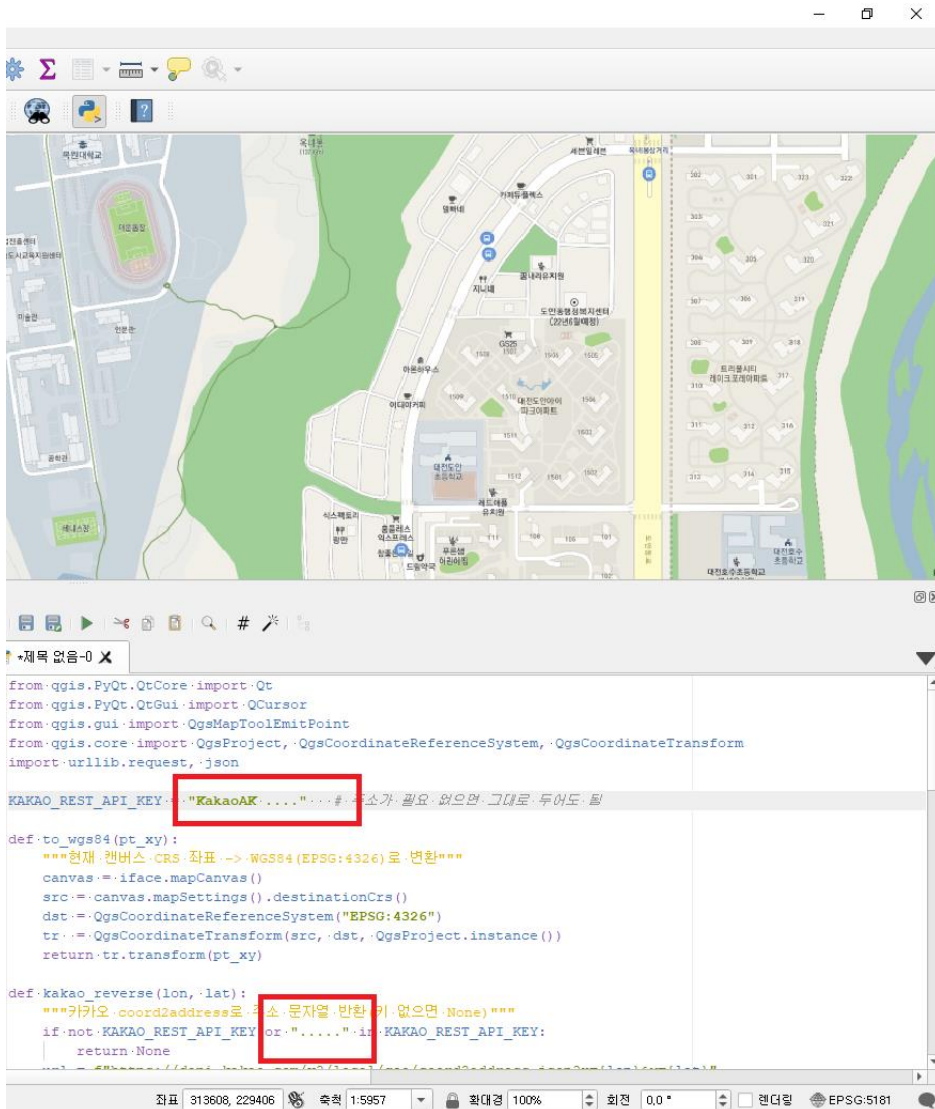
```
    if not docs:
```

```
        return None
```

```
    road = docs[0].get("road_address", {}).get("address_name")
```

```
    lot = docs[0].get("address", {}).get("address_name")
```

```
    return road or lot
```

```
class ReverseClickTool(QgsMapToolEmitPoint):
```

```
    def __init__(self, canvas):
        super().__init__(canvas)
        self.canvas = canvas
        self.prev_tool = None
```

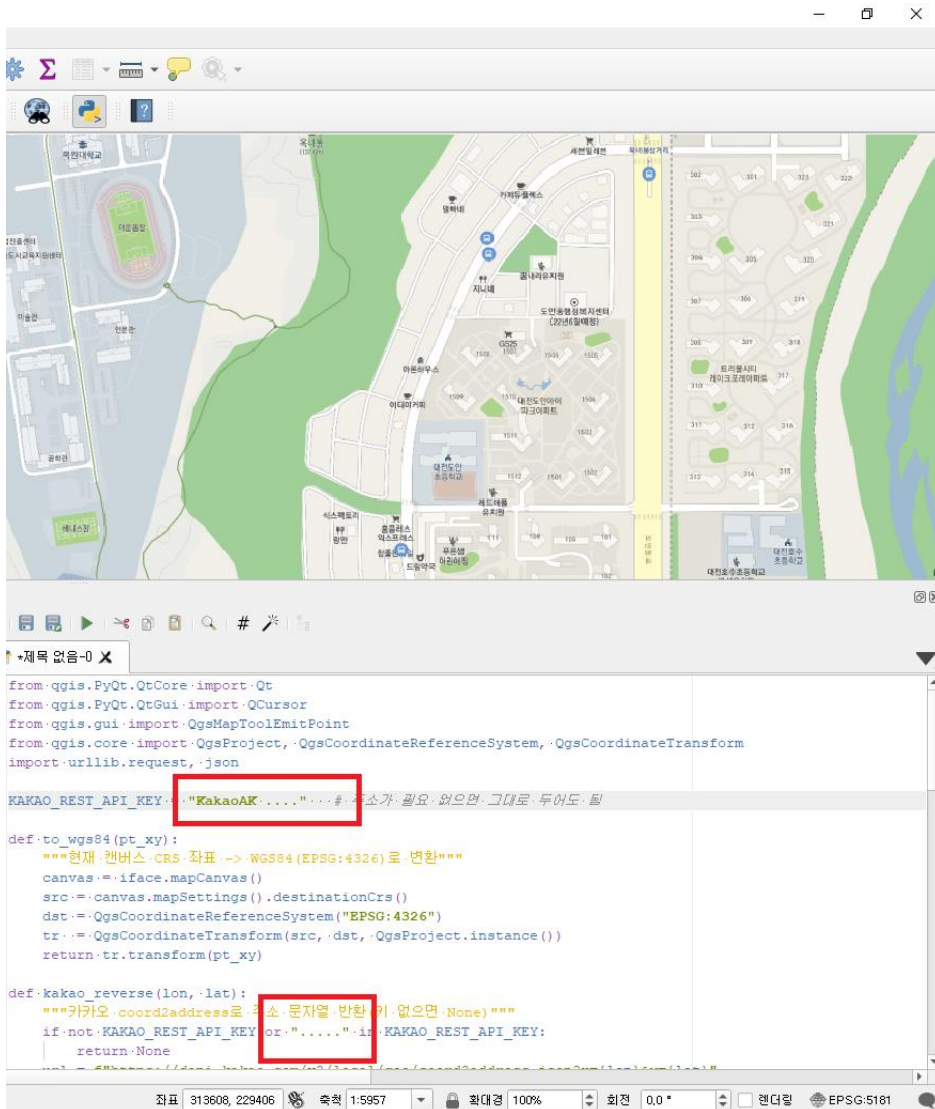
```
    def activate(self):
        self.canvas.setCursor(QCursor(Qt.CrossCursor))
```

```
    def canvasReleaseEvent(self, e):
        # 1) 클릭 지점 -> WGS84
        map_pt = self.toMapCoordinates(e.pos())
        wgs = to_wgs84(map_pt)
        lon, lat = wgs.x(), wgs.y()
```

```
        # 2) 콘솔에 위도·경도 출력
        print(f"[CLICK] lat={lat:.6f}, lon={lon:.6f}")
```

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 리버스 지오코딩



3) (옵션) 카카오 리버스 지오코딩 결과도 출력
try:

```

addr = kakao_reverse(lon, lat)
if addr:
    print(f"[ADDR] {addr}")
except Exception as ex:
    print(f"[ADDR-ERROR] {ex}")

```

try:

```

rev_tool # 이전 객체가 있으면 무시
except NameError:
    pass

```

```

rev_tool = ReverseClickTool(iface.mapCanvas())
rev_tool.prev_tool = iface.mapCanvas().mapTool()
iface.mapCanvas().setMapTool(rev_tool)

```

종료/원복:

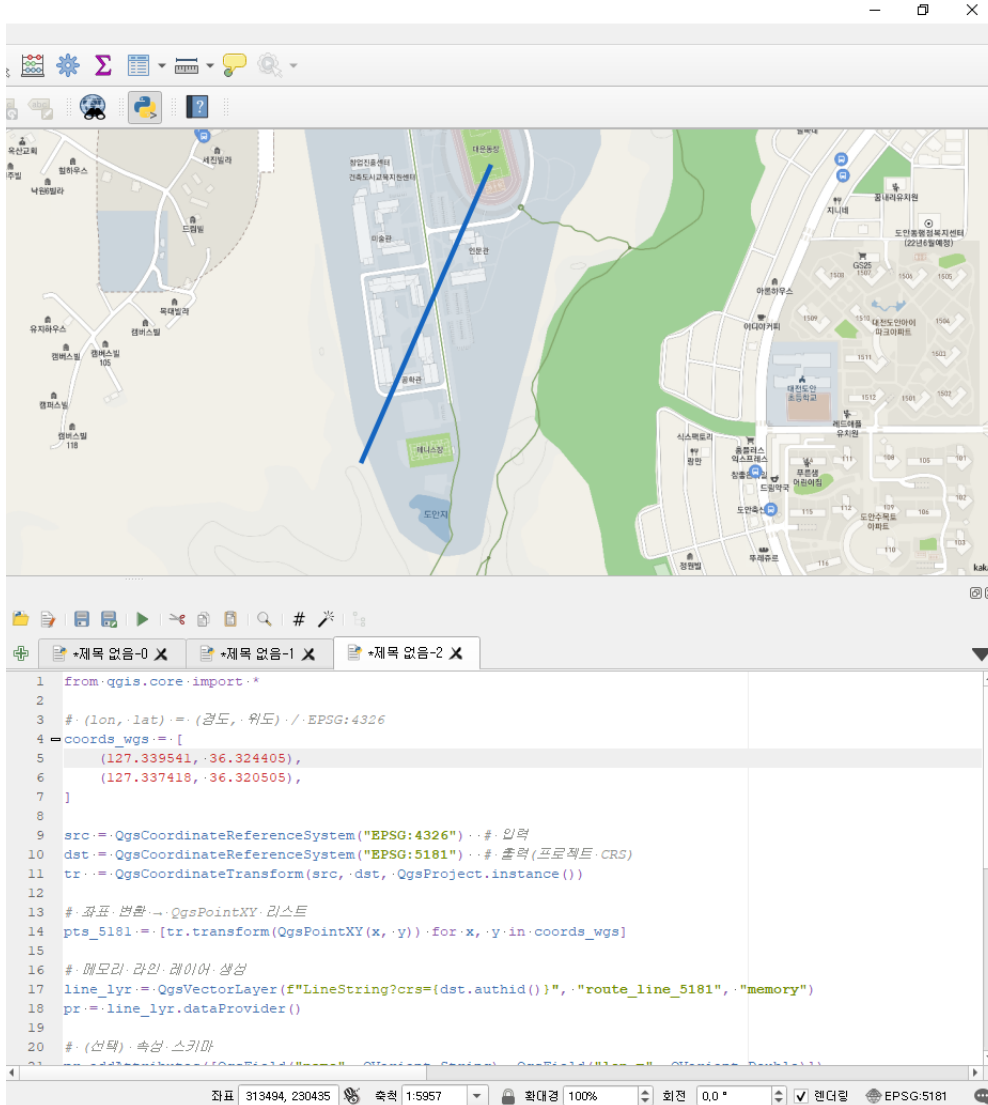
```

iface.mapCanvas().setMapTool(rev_tool.prev_tool) 실행

```

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 직선 레이어 생성



from qgis.core import *

```

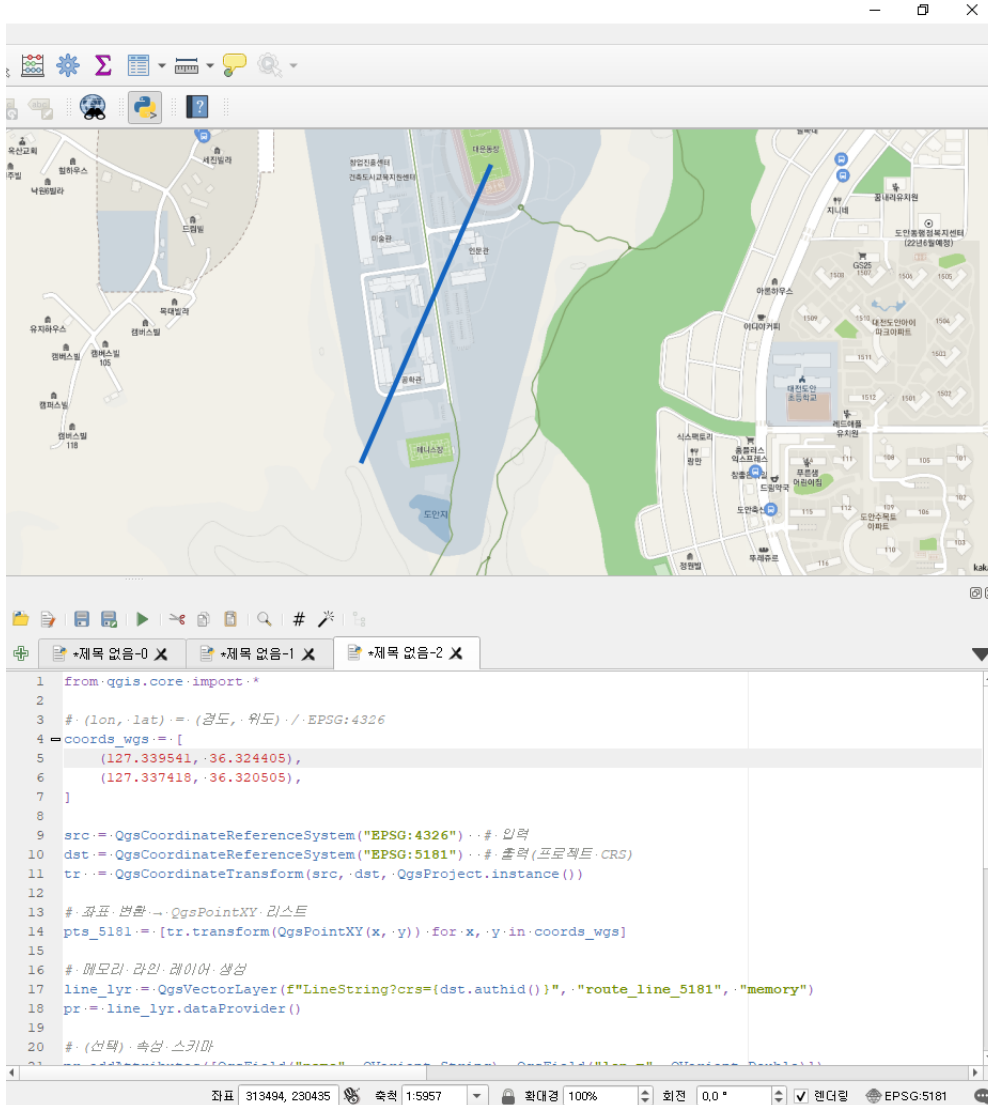
# (lon, lat) = (경도, 위도) / EPSG:4326
coords_wgs = [
    (127.339541, 36.324405),
    (127.337418, 36.320505),
]
    
```

```

src = QgsCoordinateReferenceSystem("EPSG:4326") # 입력
dst = QgsCoordinateReferenceSystem("EPSG:5181") # 출력(프로젝트 CRS)
tr = QgsCoordinateTransform(src, dst, QgsProject.instance())
    
```

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 직선 레이어 생성



좌표 변환 → QgsPointXY 리스트

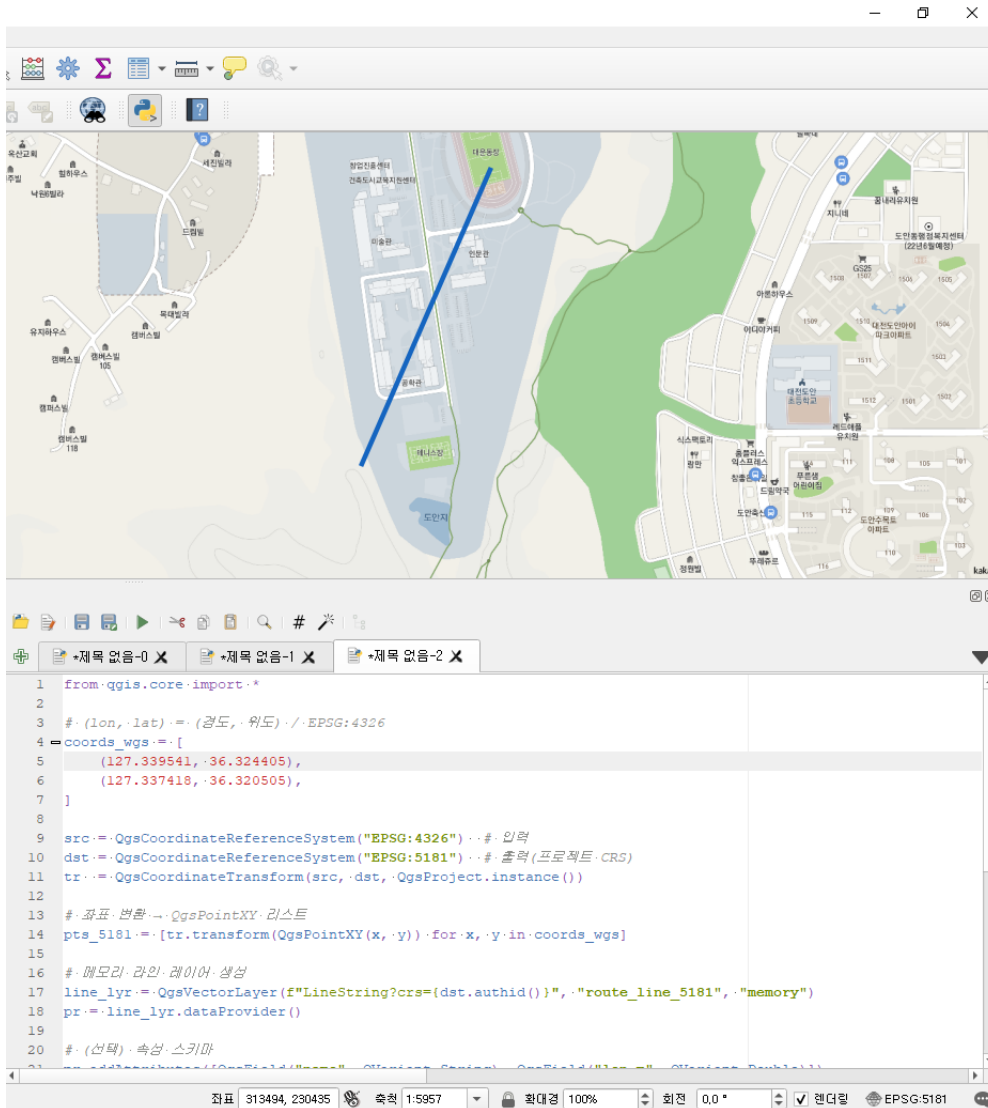
```
pts_5181 = [tr.transform(QgsPointXY(x, y)) for x, y in
coords_wgs]
```

메모리 라인 레이어 생성

```
line_lyr = QgsVectorLayer(f"LineString?crs={dst.authid()}",
"route_line_5181", "memory")
pr = line_lyr.dataProvider()
```

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 직선 레이어 생성



(선택) 속성 스키마

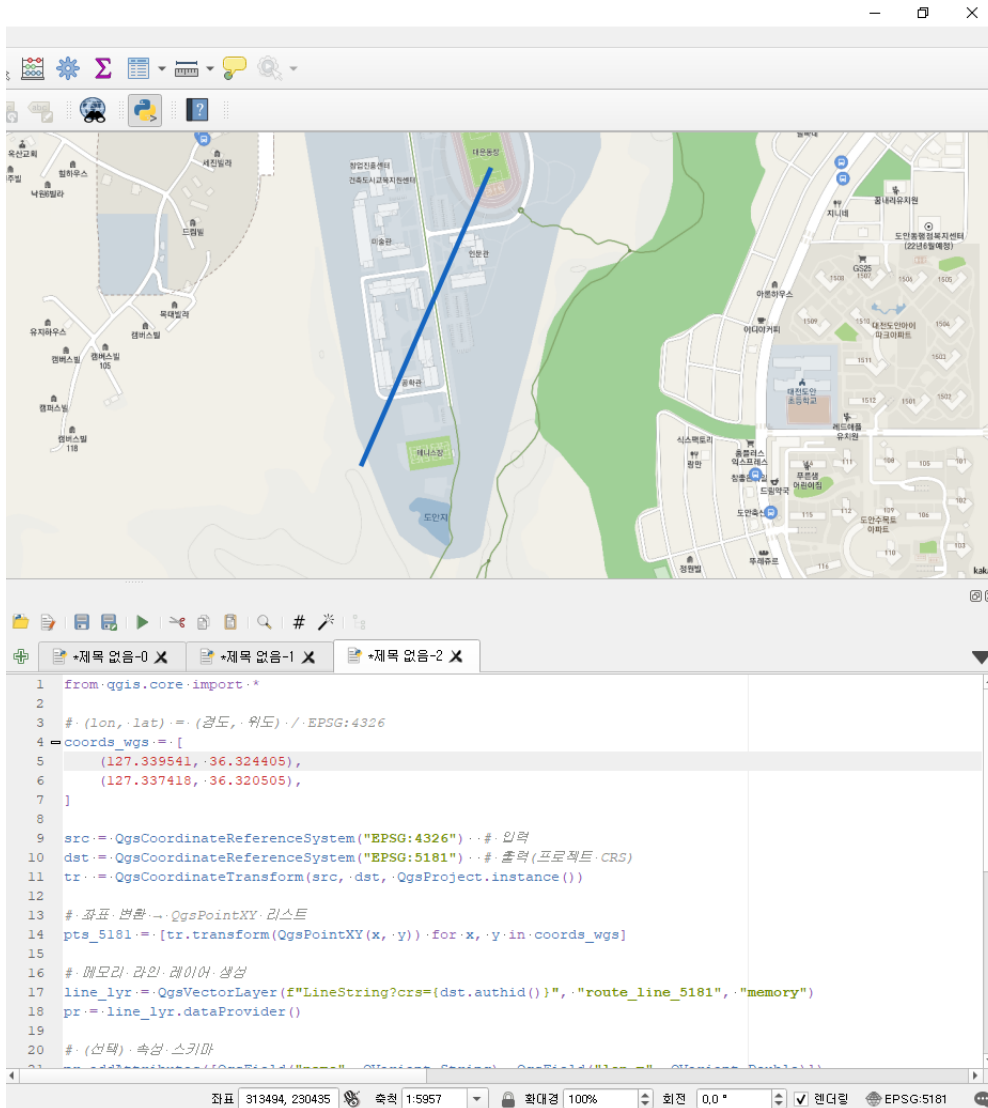
```
pr.addAttributes([QgsField("name", QVariant.String),
                  QgsField("len_m", QVariant.Double)])
line_lyr.updateFields()
```

피처 추가

```
feat = QgsFeature(line_lyr.fields())
feat.setGeometry(QgsGeometry.fromPolylineXY(pts_5181))
# 길이(m)
feat["name"] = "route-A"
feat["len_m"] = feat.geometry().length()
pr.addFeatures([feat])
```


1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 직선 레이어 생성



line_lyr.updateExtents()

QgsProject.instance().addMapLayer(line_lyr)

간단 스타일

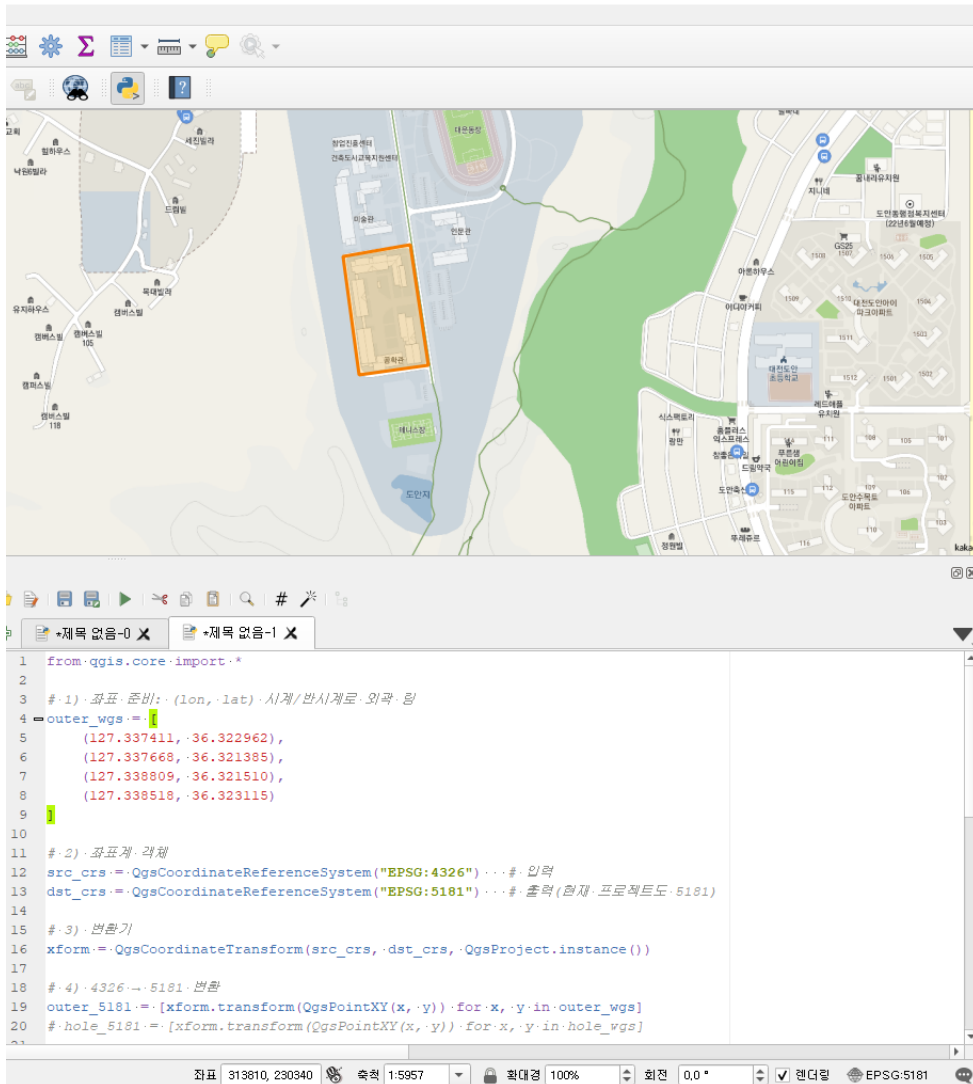
line_lyr.setRenderer(QgsSingleSymbolRenderer(QgsLineSymbol.createSimple({"color": "#1565c0", "width": "1.2"})))

line_lyr.triggerRepaint()

print("폴리라인 생성 완료 (4326→5181)")

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 폴리곤 레이어 생성



from qgis.core import *

1) 좌표 준비: (lon, lat) 시계/반시계로 외곽 링

```
outer_wgs = [
    (127.337411, 36.322962),
    (127.337668, 36.321385),
    (127.338809, 36.321510),
    (127.338518, 36.323115)
]
```

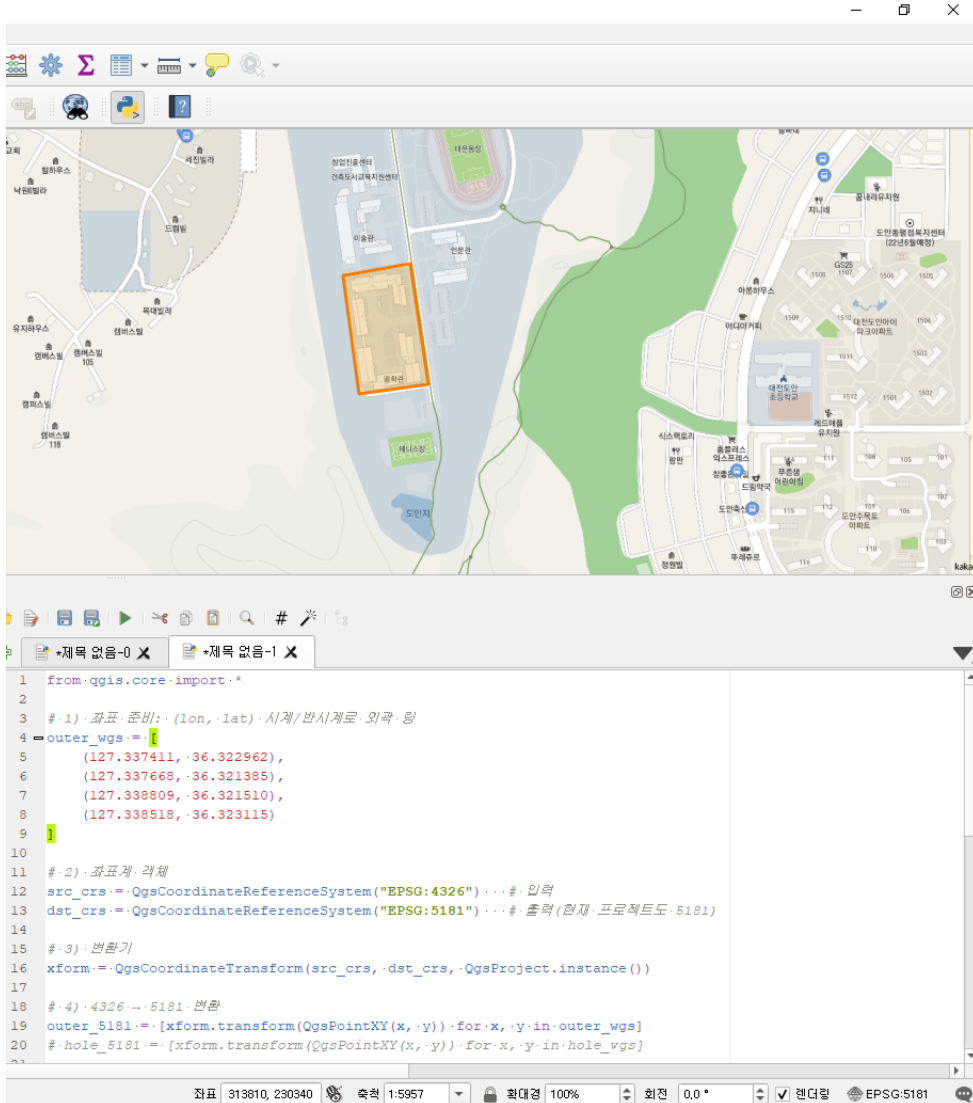
2) 좌표계 객체

```
src_crs = QgsCoordinateReferenceSystem("EPSG:4326") # 입력
```

```
dst_crs = QgsCoordinateReferenceSystem("EPSG:5181") # 출력(현재 프로젝트도 5181)
```

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 폴리곤 레이어 생성



3) 변환기

```
xform = QgsCoordinateTransform(src_crs, dst_crs,
                                QgsProject.instance())
```

4) 4326 → 5181 변환

```
outer_5181 = [xform.transform(QgsPointXY(x, y)) for x, y in
               outer_wgs]
```

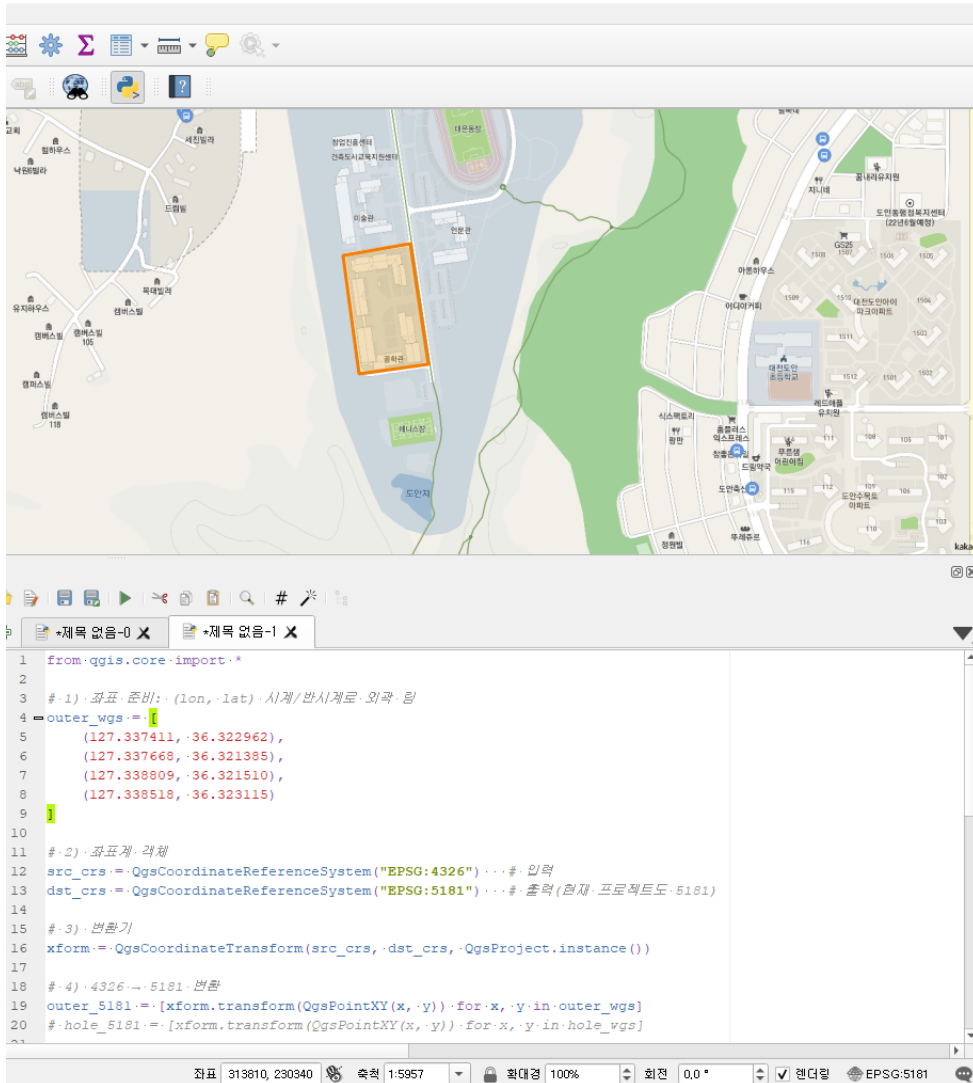
```
# hole_5181 = [xform.transform(QgsPointXY(x, y)) for x, y in
               hole_wgs]
```

5) 폴리곤 레이어(메모리) 생성 및 피처 추가

```
poly_lyr = QgsVectorLayer(f"Polygon?crs={dst_crs.authid()}",
                           "poly_5181", "memory")
pr = poly_lyr.dataProvider()
```

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 폴리곤 레이어 생성



속성(선택)

```
pr.addAttributes([QgsField("name", QVariant.String),
QgsField("area_m2", QVariant.Double)])
poly_lyr.updateFields()
```

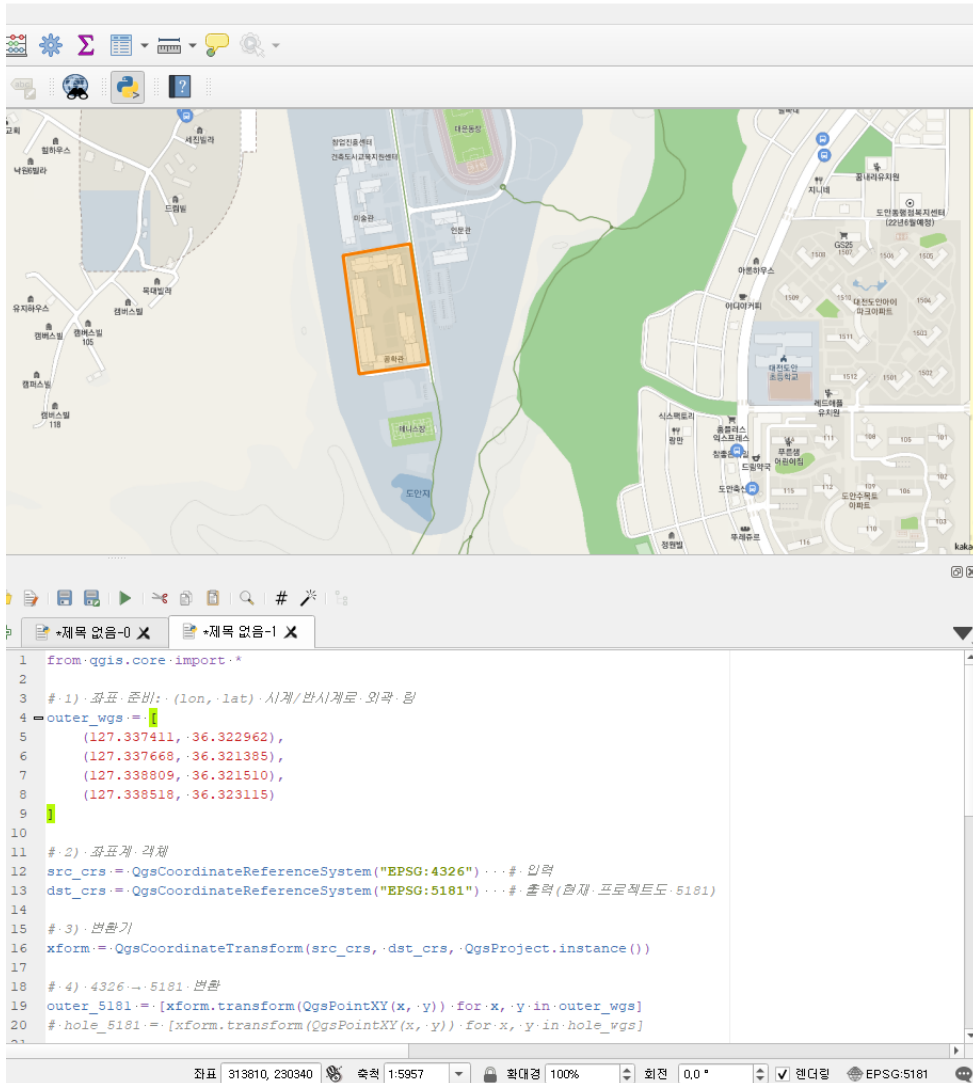
지오메트리: 외곽 + (옵션) 내부 홀

```
rings = [outer_5181] # 내부 홀이 있으면
rings.append(hole_5181)
geom = QgsGeometry.fromPolygonXY(rings)
```

```
f = QgsFeature(poly_lyr.fields())
f.setGeometry(geom)
f["name"] = "demo"
f["area_m2"] = geom.area()
pr.addFeatures([f])
```

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 폴리곤 레이어 생성



```
poly_lyr.updateExtents()
```

```
QgsProject.instance().addMapLayer(poly_lyr)
```

```
# 스타일(반투명 채움)
```

```
sym =
```

```
QgsFillSymbol.createSimple({"color":"255,165,0,60","outline_color":"#f57c00","outline_width":"0.8"})
```

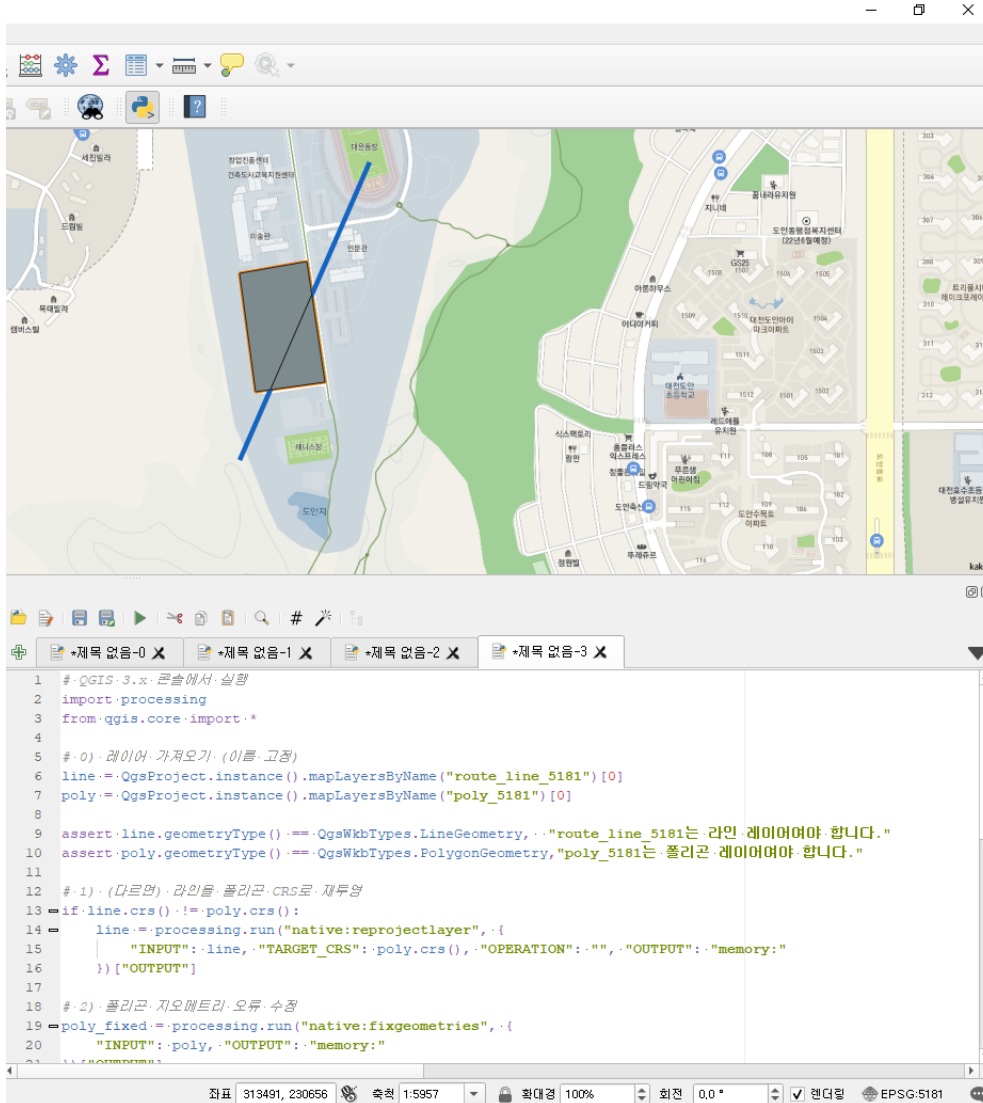
```
poly_lyr.setRenderer(QgsSingleSymbolRenderer(sym))
```

```
poly_lyr.triggerRepaint()
```

```
print("폴리곤 생성 완료 (EPSG:4326 → EPSG:5181 변환)")
```


1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 폴리곤 쪼개기



1) (다르면) 라인을 폴리곤 CRS로 재투영

if line.crs() != poly.crs():

```

    line = processing.run("native:reprojectlayer", {
        "INPUT": line, "TARGET_CRS": poly.crs(), "OPERATION":
        "", "OUTPUT": "memory:"
    })["OUTPUT"]

```

2) 폴리곤 지오메트리 오류 수정

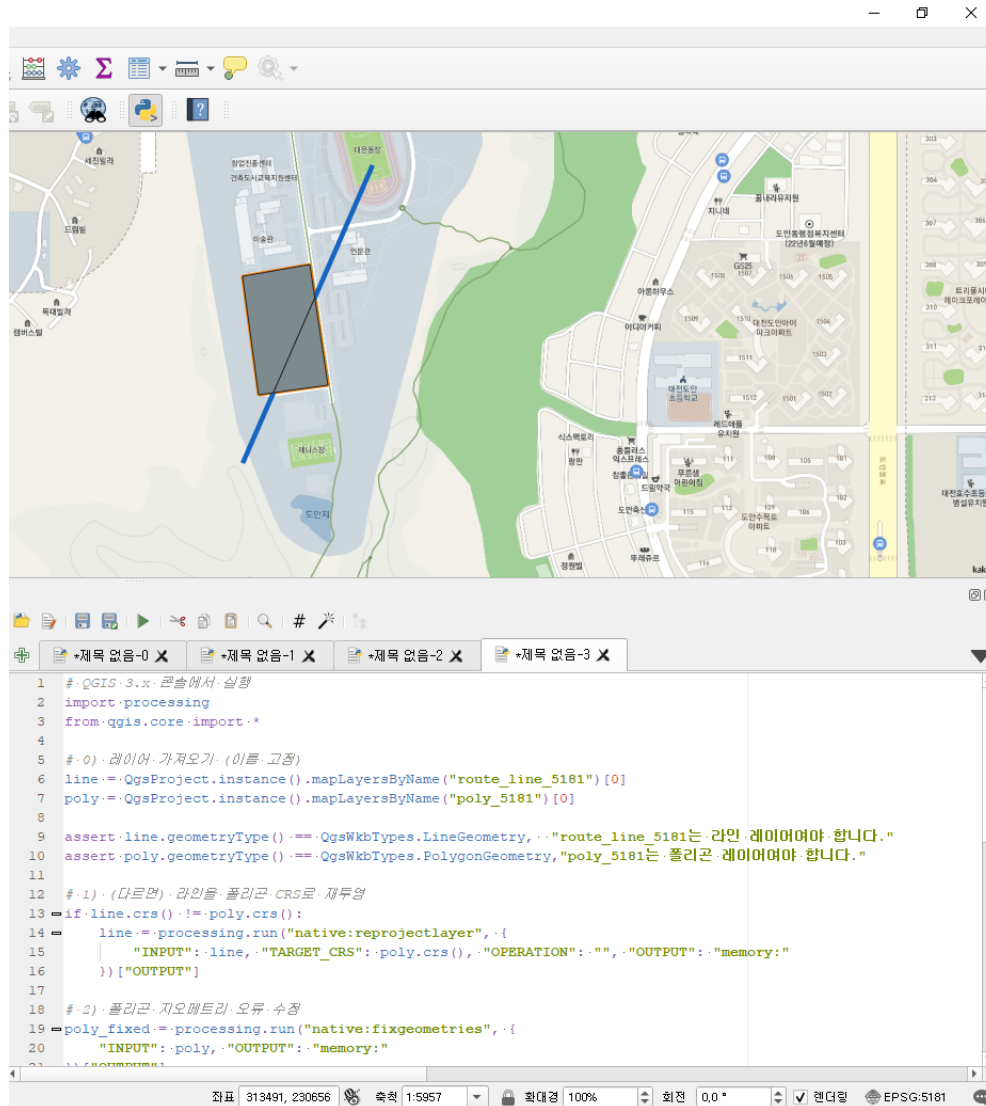
```

poly_fixed = processing.run("native:fixgeometries", {
    "INPUT": poly, "OUTPUT": "memory:"
})["OUTPUT"]

```

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 폴리곤 쪼개기



3) 라인 앞/뒤로 연장(절단 실패 방지) — 단위는 poly CRS(예: EPSG:5181이면 m)

```

extended = processing.run("native:extendlines", {
    "INPUT": line, "START_DISTANCE": 1000, "END_DISTANCE":
    1000, "OUTPUT": "memory:"
})["OUTPUT"]

```

4) 폴리곤 경계에 스냅(미세 틈새 보정)

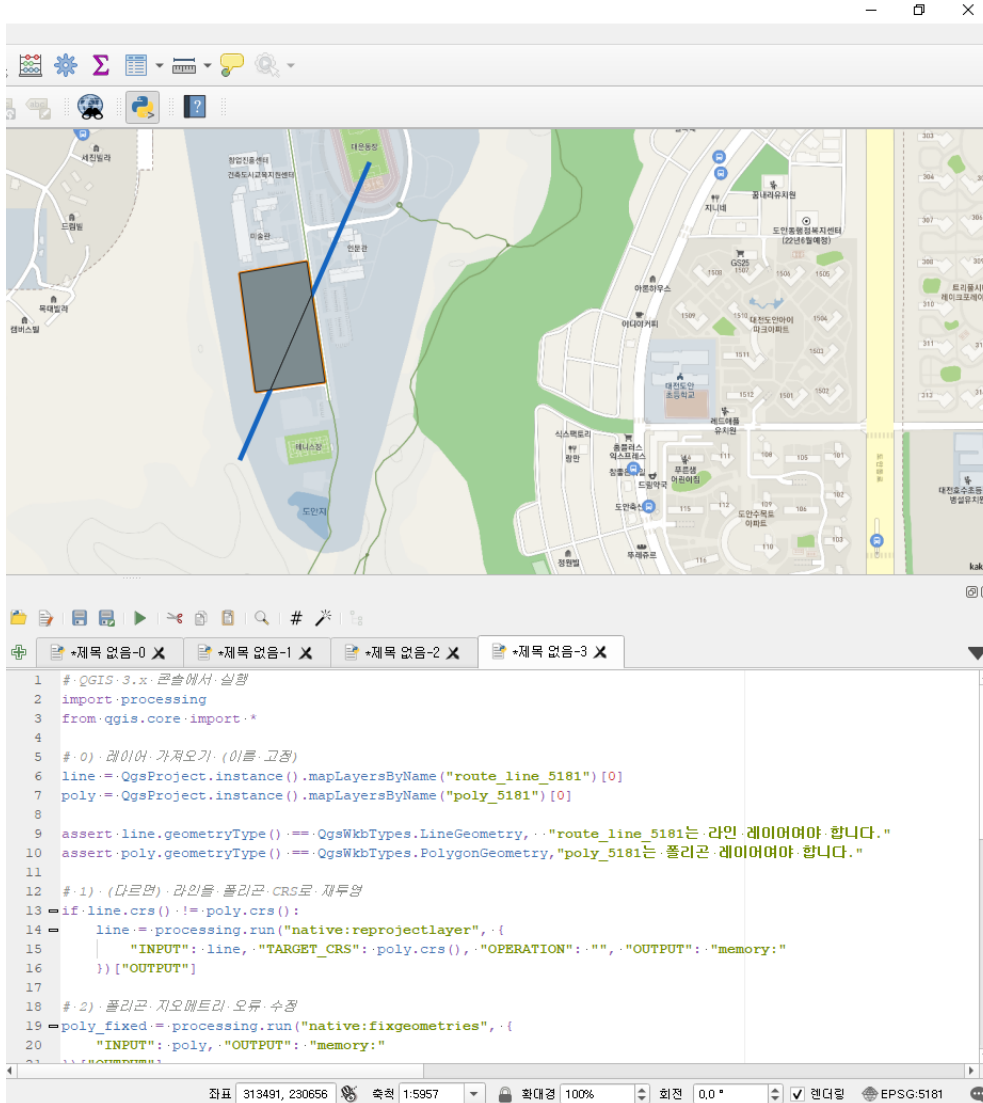
```

snapped = processing.run("native:snapgeometries", {
    "INPUT": extended,
    "REFERENCE_LAYER": poly_fixed,
    "TOLERANCE": 1.0, # 0.5m; 필요 시 1.0~2.0으로 조정
    "BEHAVIOR": 0, # 가장 가까운 곳으로
    "OUTPUT": "memory:"
})["OUTPUT"]

```

1. 공간정보 프로그래밍

파이썬으로 QGIS 핸들링하기 - 폴리곤 쪼개기



5) 라인으로 폴리곤 분할

```

split_polys = processing.run("native:splitwithlines", {
    "INPUT": poly_fixed,
    "LINES": line,
    "OUTPUT": "memory:"
})["OUTPUT"]

```

```

split_polys.setName("poly_5181_split_by_route")

```

```

QgsProject.instance().addMapLayer(split_polys)

```

print("✅ 완료: poly_5181을 route_line_5181로 분할했습니다. (결과: poly_5181_split_by_route)")

감사합니다

임 호 용

imyong7@hanmail.net
